All students should watch the videos and review the readings accompanying this LGA in preparation for this assignment and the next lecture.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

1. For each item, indicate whether it is a constant time operation or not.
   - constant / not constant : assignment of a `vector` (not just an element in the vector)

     not constant. Assigning a single element could be constant - unless that value is also variably sized, e.g., another vector.

   - constant/ not constant : accessing an array of `int` at index n

     constant

   - constant / not constant : calling a function with signature `void fn()`

     constant, if we consider just the act of invoking the function and handing control off to it, because we are not passing in or returning a variable sized object (or indeed any values at all). If we interpret "calling a function" to include the time to *execute* the function (and any functions it calls), then the cost depends on the operations performed inside the function body; it could be that the function depends on some global variable such that the "size of the problem" the function solves varies.

   - constant / not constant : addition (concatenation) of two `string` values

     not constant. In C++, concatenating two strings requires making a new `string` object and copying all of the characters from the two strings into it. So the cost is $O(m + n)$ if the strings are of length m and n.

   - constant / not constant : assignment of a `float` value

     constant

   - constant / not constant : squaring an `int`

     constant. A fixed exponent has a constant cost.

   - constant / not constant : accessing the n^{th} node in a linked list

     not constant, assuming you are just given the head of the linked list and the value n, you must follow the links between nodes $n - 1$ times in order to find the n^{th} one.

   - constant / not constant : Comparing two integers to see which is larger

     constant, assuming `int` values, not arbitrary (unbounded) integers

   - constant / not constant : dereferencing a pointer

     This is ambiguous. What type of value are we dereferencing, and what are we doing with the value? I.e., are we assigning something? If so, it really matters whether the pointer is to an `int` vs. a `vector`. The act of dereferencing itself is constant in the sense that you are

simply accessing some address in RAM.

- constant / not constant : raising an `int` to power k

  not constant, except when the `int` value is 2 and k is a non-negative integer less than the word size of your computer. We will explore exponentiation algorithms in lecture soon.

- constant / not constant : multiplication of two `double` values

  constant

- constant / not constant : printing the integer value n of arbitrary size in base 10

  not constant; each digit takes some finite time to print, and the number of digits is in $O(\log_{10} n)$.

2. Rank the following functions (from 1 to 8) in ascending order of their growth rates.

5 $n^{4/3}$
3 $\sqrt{n} \log n$
7 2^n
6 n^2
1 1
2 $\log n$
8 $n!$
4 n

Notes:

$\sqrt{n}$ grows faster than 1, so $\sqrt{n} \cdot \log n$ grows faster than $1 \cdot \log n$.

Similarly, $\sqrt{n}$ grows faster than $\log n$, thus $\sqrt{n} \cdot \sqrt{n} = n$ grows faster than $\sqrt{n} \cdot \log n$.

For $n!$, see Stirling's approximation

3. Simplify the following expressions in "big-O":
   - $O(n^3 + n + 1)$ $O(n^3)$
   - $O(\log n + n \log n - n)$ $O(n \log n)$
   - $O\left(\frac{2n^2}{3} + 100n + 42\right)$ $O(n^2)$
   - $O\left(\sum_{i=1}^{n} i\right) = O(1 + 2 + 3 + \cdots + n)$ $O\left(\frac{n(n+1)}{2}\right) = O(n^2)$
   - $O(2^n + n^3)$ $O(2^n)$
   - $O(n^{100} + n! + \sqrt{n})$ $O(n!)$

Group questions

Analyze the following functions and give an asymptotic complexity in terms of n. Assume that printing small values and short strings (using `cout`) is a constant time operation. Divide these up so that each function has at least three analyses from different group members.

1. (Basic)

```
void times_table(unsigned int n) {
    for (unsigned int i = 0; i <= n; i++) {
        for (unsigned int j = 0; j <= n; j++) {
            cout << i * j << " ";
        }
        cout << endl;
    }
}
```

The outer loop executes $n + 1$ times, and the inner loop executes $n + 1$ times for each execution of the outer loop. By assumption, the other statements are constant time operations. Therefore the complexity is $O((n + 1) \cdot (n + 1)) = O(n^2 + 2n + 1) = O(n^2)$.

2. (Intermediate)

```
void print_powers(int n) {
    int val = 1;
    int p = 0;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            cout << n << "^" << p << " = " << val << "\t";
            val = val * n;
            p++;
        }
        cout << endl;
    }
}
```

There are nested loops here, but they each run a finite number of times and do not depend on n. Regardless of n, we perform the (constant time) operations in the inner loop 25 times. This function is $O(1)$.

3. (Intermediate)

```
void aesop(unsigned int n) {
    unsigned tortoise = 0;
    unsigned hare = 0;
    while (tortoise < n && hare < n) {
        if (tortoise > hare) {
            hare += (n - tortoise) / 2;
        }
        tortoise++;
    }
    if (tortoise > hare) cout << "Tortoise wins!";
    else if (hare > tortoise) cout << "Hare wins!";
    else cout << "It's a tie!";
}
```

Whenever the tortoise gets ahead of the hare, the hare jumps ahead just short of halfway between the tortoise and the finish line. The hare is ahead for a bit, but the tortoise always plods ahead eventually, and always wins any race where $n > 0$. The tortoise's steady movement thus dictates how many times this loop executes. This is $O(n)$.

For Further Challenge

Bubblesort is a famous (infamous?) sorting algorithm mainly used in the teaching of algorithms. Its pseudo-code can be found below (note: this is different than the algorithm expressed in your textbook).

```
function BUBBLESORT(A[1..n])        // A is an array of n comparable values
    repeat
        swapped ← false
        for i = 2 to n do
            if A[i − 1] > A[i] then
                SWAP(A[i − 1], A[i])
                swapped ← true
    until not swapped
```

This algorithm performs better on some inputs than others. Can you describe a *best case* input, on which the algorithm performs the fewest comparisons? What is the complexity of BUBBLESORT in the best case? Next, can you describe a *worst case* input, and analyze the algorithm in the worst case?

In the best case, the *swapped* variable is never true. This can only happen if the input is already sorted. In this case, the algorithm performs one pass over the data, doing $n - 1$ comparisons to verify that the array is already sorted. The complexity in the best case is thus $O(n)$.

To understand the worst case, we have to first recognize how this sorting algorithm works. It loops over the input repeatedly, and on each pass it simply compares adjacent neighbors, swapping them if they are out of place relative to each other.

With this algorithm (as expressed above), note that elements can move "up" towards the back of the array many times through successive swaps. For example, if we have the array [5, 1, 2, 3, 4], the 5 will actually move into the correct position in just one pass (one additional pass is necessary to determine the array is sorted).

On the other hand, no element can move towards the front of the array more than one place on any pass. This gives us our worst case: an array such that the minimum element is at the end of the array, e.g., [2, 3, 4, 5, 1]. (You can make an even worse case with a completely reversed array; the number of comparisons doesn't increase, but the number of swaps does.) In this case we require $n - 1$ passes to get the minimum element in the correct location, and one more pass to verify that the array is sorted. Each pass performs $n - 1$ comparisons. Thus the total work is $O(n(n - 1)) = O(n^2)$.

We will also discuss *average case* analysis, in which we consider the average performance over some distribution of inputs. If we assume our inputs consist of distinct elements and that any arrangement of the elements is equally probable, then it is straightforward to show that the average case is at least $O(n^2)$ for bubblesort.